

02_geracao_base_qna

September 29, 2026

1 Geração da base de QnA da Indie com LangChain

Projeto final: Generative AI & Advanced Analytics

Este notebook constrói a base de perguntas e respostas usada no fine-tuning do assistente virtual da **Indie**, fintech fictícia para quem trabalha por conta própria. O documento `01_empresa_indie.md` é a **fonte da verdade**: todo par gerado precisa ser fiel a ele.

1.1 Metodologia

1. **Geração guiada por matriz tema × estilo.** A base é dividida em 24 temas (produtos, planos, suporte, segurança, limites da assistente etc.) e 6 estilos de pergunta (direta, comparação, suporte, caso de uso, objeção e informal). Cada combinação gera 5 pares com o `gpt-4o-mini`, via proxy da disciplina, usando `ChatPromptTemplate` e `with_structured_output`.
2. **Anti-repetição.** Cada chamada recebe as perguntas já geradas para o mesmo tema e é instruída a não repeti-las. Também recebe personas aleatórias (psicóloga MEI, dev que recebe em dólar etc.) para diversificar os cenários.
3. **Limpeza automática.** Normalização de texto, filtros de tamanho, remoção de duplicatas exatas e de quase-duplicatas (similaridade TF-IDF entre perguntas).
4. **Checação de números.** Valores em R\$ e percentuais das respostas que não aparecem no documento são sinalizados para revisão.
5. **LLM como juiz.** Um segundo passe, com temperatura 0, compara cada par com o documento e descarta respostas inconsistentes, vagas ou fora das diretrizes.
6. **Revisão manual de 10%.** Uma amostra estratificada por tema é exportada para planilha, revisada pelo grupo e reaplicada à base.
7. **Exportação** no formato `.jsonl` com a estrutura `messages` (system/user/assistant), igual ao dataset usado em aula.

Todos os passos que chamam o LLM salvam checkpoints. Se o Colab desconectar ou o token do proxy expirar, basta refazer o login e reexecutar a célula: o trabalho já feito não é repetido.

1.2 1. Preparação do ambiente

A célula abaixo instala o cliente de autenticação do proxy (`pgl-auth`), a integração do LangChain com APIs compatíveis com a OpenAI e as bibliotecas de apoio.

```
[30]: !uv pip install pgl-auth langchain-openai langchain-core pydantic scikit-learn
      ↪ tqdm -qqq
```

A célula abaixo importa as bibliotecas usadas ao longo do notebook.

```
[31]: import json
import os
import random
import re
import time
from pathlib import Path
from typing import Any, Optional

import numpy as np
import pandas as pd
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from tqdm.auto import tqdm
```

1.2.1 1.1 Configuração

A classe `Config` centraliza caminhos e parâmetros da geração. Com `USE_GOOGLE_DRIVE = True`, os arquivos ficam no Google Drive e sobrevivem a uma desconexão do Colab. Recomendamos deixar assim.

Parâmetro	Valor	Justificativa
GENERATOR_TEMPERATURE	0.8	Mais diversidade nas perguntas; erros factuais são filtrados pelo juiz
PAIRS_PER_CALL	5	Lotes pequenos mantêm a qualidade de cada par e ainda somam cerca de 700 pares brutos
DEDUP_THRESHOLD	0.85	Similaridade de cosseno (n-gramas de caracteres) acima da qual duas perguntas são consideradas quase idênticas
JUDGE_TEMPERATURE	0.0	Avaliação determinística
REVIEW_FRACTION	0.10	Amostra para revisão manual, conforme o enunciado

```
[32]: class Config:
      """Central configuration for the QnA dataset generation.
```

Groups file locations, proxy/model settings and quality-control thresholds in a single place, making the pipeline easier to reproduce and adjust.

"""

```
USE_GOOGLE_DRIVE = True
DRIVE_DIR = "/content/drive/MyDrive/PUC_GenAI_Indie"
LOCAL_DIR = "indie_qna"

COMPANY_DOC_FILE = "01_empresa_indie.md"
RAW_FILE = "qna_bruto.jsonl"
JUDGE_FILE = "qna_avaliacao_juiz.jsonl"
REVIEW_FILE = "amostra_revisao_manual.csv"
REVIEWED_FILE = "amostra_revisao_manual_revisada.csv"
METADATA_FILE = "indie_qna_com_metadados.jsonl"
FINAL_FILE = "indie_knowledge_base.jsonl"

PROXY_BASE_URL = "https://pgl-proxy.vercel.app/v1"
GENERATOR_MODEL = "gpt-4o-mini"
GENERATOR_TEMPERATURE = 0.8
JUDGE_MODEL = "gpt-4o-mini"
JUDGE_TEMPERATURE = 0.0
# Troque para "function_calling" se o proxy recusar "json_schema".
STRUCTURED_METHOD = "json_schema"

PAIRS_PER_CALL = 5
MAX_CONCURRENCY = 6
MAX_RETRIES = 3
MAX_EXISTING_QUESTIONS = 40
PERSONAS_PER_CALL = 3

MIN_QUESTION_CHARS = 10
MIN_ANSWER_CHARS = 60
DEDUP_THRESHOLD = 0.85
JUDGE_BATCH_SIZE = 10
REVIEW_FRACTION = 0.10
TARGET_MIN_EXAMPLES = 500

SEED = 42

def __init__(self) -> None:
    self.base_dir = Path(
        self.DRIVE_DIR if self.USE_GOOGLE_DRIVE else self.LOCAL_DIR
    )

def path(self, filename: str) -> Path:
    """Return the full path of a project file inside the base dir."""
```

```
return self.base_dir / filename
```

A célula abaixo monta o Google Drive (se habilitado), cria a pasta do projeto e inicializa o dicionário `pipeline_stats`. Ele acumula os números de cada etapa para o relatório final.

```
[33]: config = Config()

if config.USE_GOOGLE_DRIVE:
    from google.colab import drive

    drive.mount("/content/drive")

config.base_dir.mkdir(parents=True, exist_ok=True)
random.seed(config.SEED)
pipeline_stats: dict[str, Any] = {}

print(f"Arquivos do projeto em: {config.base_dir}")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

Arquivos do projeto em: /content/drive/MyDrive/PUC_GenAI_Indie

1.3 2. Autenticação no proxy da disciplina

Seguindo o notebook `pgl_auth_documented.ipynb`, a matrícula e a senha são lidas dos *secrets* do Colab (`PGL_REGISTRATION_NUMBER` e `PGL_PASSWORD`). Fora do Colab, a função `get_secret` usa variáveis de ambiente com o mesmo nome.

```
[34]: def get_secret(name: str) -> str:
    """Fetch a secret from Colab secrets or environment variables.

    Args:
        name: The secret name (e.g. 'PGL_PASSWORD').

    Returns:
        The secret value as a string.

    Raises:
        ValueError: If the secret cannot be found.
    """
    try:
        from google.colab import userdata

        value = userdata.get(name)
    except Exception:
        value = os.environ.get(name)
    if not value:
        raise ValueError(
```

```

        f"Secret '{name}' nao encontrado nos secrets do Colab "
        "nem nas variaveis de ambiente."
    )
    return value

```

A célula abaixo registra a matrícula (se ainda não houver senha cadastrada) e faz o login, obtendo o token JWT usado como `api_key` do proxy. **Se o token expirar durante a geração, reexecute esta célula e a seguinte.**

```

[35]: from pgl_auth import PGLAuthClient

auth_client = PGLAuthClient()
try:
    auth_client.register(
        registration_number=get_secret("PGL_REGISTRATION_NUMBER"),
        password=get_secret("PGL_PASSWORD"),
    )
except Exception as e:
    print(e)

token = auth_client.login(
    registration_number=get_secret("PGL_REGISTRATION_NUMBER"),
    password=get_secret("PGL_PASSWORD"),
)
print("Login realizado com sucesso.")

```

Essa matrícula já possui uma senha cadastrada.
Login realizado com sucesso.

A célula abaixo cria dois clientes do modelo: o **gerador**, com temperatura alta para dar diversidade, e o **juiz**, com temperatura 0 para uma avaliação determinística. Por fim, faz uma chamada de teste.

```

[36]: generator_llm = ChatOpenAI(
    base_url=config.PROXY_BASE_URL,
    api_key=token,
    model=config.GENERATOR_MODEL,
    temperature=config.GENERATOR_TEMPERATURE,
    timeout=180,
    max_retries=2,
)
judge_llm = ChatOpenAI(
    base_url=config.PROXY_BASE_URL,
    api_key=token,
    model=config.JUDGE_MODEL,
    temperature=config.JUDGE_TEMPERATURE,
    timeout=180,
    max_retries=2,
)

```

```
print(generator_llm.invoke("Responda apenas: ok").content)
```

Ok.

1.4 3. Documento da empresa (fonte da verdade)

A função `load_company_doc` procura o arquivo `01_empresa_indie.md` na pasta do projeto. Se não encontrar, abre o upload do Colab e salva o arquivo nessa pasta. A função `extract_system_prompt` extrai o prompt de sistema da seção 7 do documento, o mesmo que será usado no dataset e no fine-tuning.

```
[37]: def load_company_doc(config: Config) -> str:
    """Load the company reference document, uploading it if needed.

    Args:
        config: The pipeline configuration.

    Returns:
        The document content as a string.
    """
    for candidate in (
        config.path(config.COMPANY_DOC_FILE),
        Path(config.COMPANY_DOC_FILE),
    ):
        if candidate.exists():
            return candidate.read_text(encoding="utf-8")

    from google.colab import files

    print(f"Envie o arquivo {config.COMPANY_DOC_FILE}:")
    uploaded = files.upload()
    content = next(iter(uploaded.values()))
    target = config.path(config.COMPANY_DOC_FILE)
    target.write_bytes(content)
    return target.read_text(encoding="utf-8")

def extract_system_prompt(company_doc: str) -> str:
    """Extract the system prompt code block from section 7 of the doc.

    Args:
        company_doc: The full company reference document.

    Returns:
        The system prompt text used in the dataset and fine-tuning.

    Raises:
        ValueError: If section 7 or its code block cannot be found.
```

```

"""
parts = company_doc.split("## 7.", 1)
if len(parts) < 2:
    raise ValueError("Secao '## 7.' nao encontrada no documento.")
match = re.search(r"```\s*\n(?:.*?)\n```", parts[1], flags=re.DOTALL)
if match is None:
    raise ValueError("Bloco de codigo com o prompt de sistema nao_
↳encontrado.")
return match.group(1).strip()

```

```

[38]: COMPANY_DOC = load_company_doc(config)
SYSTEM_PROMPT = extract_system_prompt(COMPANY_DOC)

print(f"Documento carregado: {len(COMPANY_DOC):,} caracteres.")
print("\nPrompt de sistema do dataset:\n")
print(SYSTEM_PROMPT)

```

Documento carregado: 25,300 caracteres.

Prompt de sistema do dataset:

Você é a assistente virtual da Indie, o banco de quem trabalha por conta própria. A Indie é uma fintech brasileira que oferece serviços financeiros para MEIs, microempresas de serviços, freelancers e profissionais liberais.

Produtos da Indie:

- Indie Conta PJ: conta digital para pessoa jurídica, com Pix, cartões e Indie Abre MEI
- Indie Notas: emissão de nota fiscal de serviço (NFS-e)
- Indie Global: recebimento de pagamentos do exterior em USD, EUR e GBP
- Cofre Fiscal: reserva automática para impostos e DAS-MEI automático
- Indie Contábil: contabilidade online com contador dedicado
- Indie Futuro: Caixinhas, Salário Indie, Seguro Renda Protegida e Previdência
- Indie Cobranças: links de pagamento, cobrança recorrente e Tap to Pay

Planos: Essencial (grátis), Pro (R\$ 39,90/mês) e Premium (R\$ 99,90/mês).

Responda em português, de forma clara, cordial e precisa. Não dê recomendações individuais de investimento nem planejamento tributário personalizado, nunca peça senhas ou códigos e, se não souber uma informação, oriente o cliente a falar com o atendimento.

1.5 4. Plano de geração: temas, estilos e personas

A diversidade da base vem de três eixos:

- **Temas (TOPICS):** cobrem todas as seções do documento, inclusive o que a Indie *não* faz, os limites da assistente e as perguntas fora do escopo. Assim o modelo também aprende a

recusar e a redirecionar.

- **Estilos (STYLES)**: variam o formato da pergunta, conforme pede o enunciado (diretas, comparações, suporte, casos de uso, objeções e mensagens informais).
- **Personas (PERSONAS)**: perfis de clientes sorteados a cada chamada para variar os cenários.

```
[39]: TOPICS: list[dict[str, Any]] = [
    {
        "id": "empresa",
        "name": "A empresa Indie",
        "focus": "História e fundação, fundador, missão, visão, valores, "
        "posicionamento de marca, tom de voz, público-alvo, número de clientes "
        "e principais diferenciais.",
    },
    {
        "id": "regulacao",
        "name": "Regulação e segurança do dinheiro",
        "focus": "Instituição de pagamento autorizada pelo Banco Central, "
        "segregação dos recursos, saldo da conta sem cobertura do FGC, "
        "Caixinhas cobertas pelo FGC e parceiros (Banco Horizonte, Ponte "
        "Câmbio DTVM, Aurora Seguros, Indie Contabilidade).",
    },
    {
        "id": "conta_abertura",
        "name": "Abertura da Indie Conta PJ",
        "focus": "Quem pode abrir conta, tipos de empresa aceitos, documentos, "
        "tempo de abertura, prazo de análise, requisitos do titular e quem a "
        "Indie não atende.",
    },
    {
        "id": "conta_funcionalidades",
        "name": "Funcionalidades da Indie Conta PJ",
        "focus": "Pix (chaves, agendado, Automático, Cobrança), TED, pagamento "
        "de boletos, extrato e formatos de exportação, Open Finance, usuários "
        "adicionais e o fato de o saldo parado não render.",
    },
    {
        "id": "cartoes",
        "name": "Cartões de débito e crédito PJ",
        "focus": "Cartão virtual e físico, prazo de entrega, uso no exterior, "
        "cartão de crédito com limite baseado no faturamento e liberação após "
        "60 dias, anuidade por plano, cashback do Premium, 2ª via e bloqueio.",
    },
    {
        "id": "abre_mei",
        "name": "Indie Abre MEI",
        "focus": "Abertura gratuita de MEI pelo app para quem ainda não tem "
        "CNPJ, tempo de abertura, condição de a atividade ser permitida ao MEI "
```

```

    "e a promoção de 3 meses de plano Pro.",
  },
  {
    "id": "notas",
    "name": "Indie Notas",
    "focus": "Emissão de NFS-e, Emissor Nacional para MEI, integração com "
    "prefeituras para ME, notas recorrentes, envio automático, conciliação "
    "com a cobrança, cancelamento, nota para tomador no exterior, limites "
    "por plano e certificado digital A1.",
  },
  {
    "id": "global",
    "name": "Indie Global",
    "focus": "Recebimento do exterior em USD, EUR e GBP, dados de "
    "recebimento, conversão automática ou manual por plano, prazo de "
    "crédito em reais, spread por plano, IOF, ausência de tarifa fixa, "
    "contrato de câmbio, limite mensal e o fato de não enviar dinheiro "
    "para o exterior.",
  },
  {
    "id": "cofre_fiscal",
    "name": "Cofre Fiscal",
    "focus": "Reserva automática de uma porcentagem de cada recebimento, "
    "sugestão de alíquota, rendimento, DAS-MEI automático, DAS de ME, "
    "alertas de limite do MEI, Simulador de Fator R e diferenças por plano.
↪",
  },
  {
    "id": "contabil_mei",
    "name": "Indie Contábil para MEI",
    "focus": "DASN-SIMEI (prazo, preço avulso e inclusão por plano), apoio "
    "contábil ao MEI e desenquadramento de MEI para ME.",
  },
  {
    "id": "contabil_me",
    "name": "Indie Contábil para ME/EPP",
    "focus": "Serviços incluídos na assinatura, preço por plano, "
    "funcionários, abertura de ME gratuita com fidelidade, troca de "
    "contador, atendimento do contador e regimes não atendidos.",
  },
  {
    "id": "caixinhas",
    "name": "Caixinhas",
    "focus": "CDBs com liquidez diária, cobertura do FGC, rendimento por "
    "plano, aplicação mínima, Caixinhas por objetivo, Guardar troco e "
    "tributação/informe de rendimentos.",
  },
},

```

```

{
  "id": "salario_indie",
  "name": "Salário Indie",
  "focus": "Transferência de valor fixo mensal da conta PJ para a conta "
  "pessoal, Caixinha Colchão, indicação de meses cobertos pela reserva, "
  "relação com o pró-labore e planos em que está disponível.",
},
{
  "id": "seguro",
  "name": "Seguro Renda Protegida",
  "focus": "Diária por afastamento por doença ou acidente, valores de "
  "diária, preço, carência, franquia, duração máxima por evento, "
  "abertura de sinistro e seguradora parceira.",
},
{
  "id": "previdencia",
  "name": "Previdência Indie",
  "focus": "PGBL e VGBL, aporte mínimo, taxa de administração, ausência "
  "de taxa de carregamento, tabelas de IR, portabilidade e seguradora "
  "parceira.",
},
{
  "id": "cobrancas",
  "name": "Indie Cobranças",
  "focus": "Link de pagamento, cobrança recorrente, lembretes, "
  "conciliação, Tap to Pay e requisitos do celular, ausência de "
  "maquininha física, taxas de boleto e cartão, prazo de recebimento e "
  "antecipação.",
},
{
  "id": "planos",
  "name": "Planos e preços",
  "focus": "Comparação entre Essencial, Pro e Premium: mensalidades, "
  "plano anual, TED, boletos, notas, spread, Caixinhas, cartão, Salário "
  "Indie, Tap to Pay, usuários adicionais e atendimento. Inclua "
  "perguntas sobre qual plano atende melhor a um perfil de uso.",
},
{
  "id": "regras_comerciais",
  "name": "Regras comerciais",
  "focus": "Teste grátis de 30 dias, promoção Abre MEI, troca de plano "
  "(upgrade e downgrade), cancelamento e reembolso, cobrança da "
  "mensalidade e falta de saldo, e Indie Contábil como assinatura "
  "separada.",
},
{
  "id": "atendimento",

```

```

    "name": "Canais de atendimento",
    "focus": "Chat 24h, WhatsApp, gerente de relacionamento, e-mail, "
    "central de bloqueio, SAC e Ouvidoria: horários, prazos e planos que "
    "dão acesso a cada canal.",
  },
  {
    "id": "suporte",
    "name": "Problemas e situações de suporte",
    "focus": "Pix enviado errado ou golpe (MED), compra não reconhecida "
    "(chargeback), perda ou roubo do celular, 2ª via do cartão, "
    "encerramento da conta e portabilidade.",
  },
  {
    "id": "seguranca",
    "name": "Segurança e privacidade",
    "focus": "Login com biometria, Indie Chave, troca de celular, limites "
    "de Pix e limite noturno, alertas, golpes comuns (falsa central, "
    "'conta de segurança'), LGPD, encarregado de dados e certificações.",
  },
  {
    "id": "limites_assistente",
    "name": "Limites da assistente virtual",
    "focus": "Pedidos de recomendação individual de investimento, "
    "planejamento tributário personalizado, consultas de saldo, extrato "
    "ou status de transações do próprio cliente, situações em que alguém "
    "oferece ou pede senha e códigos, e perguntas cuja resposta não está "
    "no documento. A resposta deve recusar ou redirecionar com educação, "
    "explicando o que a Indie oferece e qual canal procurar.",
  },
  {
    "id": "nao_oferece",
    "name": "O que a Indie não oferece",
    "focus": "Envio de dinheiro ao exterior, maquininha física, conta "
    "pessoa física, atendimento a empresas do Lucro Presumido/Real, "
    "empresas com mais de 5 sócios, ONGs, cooperativas e associações, e "
    "pedidos de comparação com bancos concorrentes (sem falar mal deles). "
    "Mencione o roadmap quando existir.",
  },
  {
    "id": "fora_escopo",
    "name": "Perguntas fora do escopo",
    "focus": "Perguntas sem relação com a Indie (culinária, esportes, "
    "política, pedidos de código de programação, curiosidades gerais). A "
    "resposta deve recusar de forma educada e breve e oferecer ajuda com "
    "os temas da Indie.",
    "styles": ["direta", "informal"],
  },
},

```

```

]

STYLES: dict[str, str] = {
    "direta": (
        "Perguntas diretas e objetivas sobre fatos: o que é, quanto custa, "
        "qual o prazo, como funciona, quem pode usar."
    ),
    "comparacao": (
        "Pedidos de comparação ou de diferença: entre planos, entre produtos "
        "da Indie, entre opções de um mesmo produto ou entre conceitos (ex.: "
        "Cofre Fiscal x Caixinhas)."
    ),
    "suporte": (
        "Dúvidas de quem já é cliente: passo a passo ('como faço para...'), "
        "problemas práticos ('não consigo...', 'apareceu...') e consequências "
        "('o que acontece se...')."
    ),
    "caso_de_uso": (
        "Cenários concretos em primeira pessoa, em que o cliente descreve a "
        "profissão e a situação (valores, rotina, dificuldade) e pergunta se "
        "ou como a Indie resolve. Use as personas sugeridas."
    ),
    "objecao": (
        "Objeções e desconfiças de quem ainda não é cliente: segurança, "
        "custos escondidos, 'por que não usar meu banco atual', 'qual é a "
        "pegadinha', 'e se a Indie quebrar'."
    ),
    "informal": (
        "Mensagens curtas e informais, como no WhatsApp: linguagem coloquial, "
        "abreviações (vc, pq, qto, tb), às vezes sem pontuação ou com pequenos "
        "erros de digitação. A resposta continua cordial, correta e completa."
    ),
}

PERSONAS: list[str] = [
    "psicóloga MEI que atende pacientes online",
    "desenvolvedor freelancer que recebe em dólar de uma empresa dos EUA",
    "designer gráfica MEI em início de carreira",
    "personal trainer com alunos mensalistas",
    "tradutora que recebe em euro de clientes europeus",
    "sócio de uma agência de marketing ME com 3 sócios",
    "fotógrafo de casamentos com renda muito variável",
    "nutricionista com consultório próprio",
    "consultor de TI com ME no Simples Nacional",
    "criadora de conteúdo que recebe de marcas",
    "professor particular de inglês",
    "fisioterapeuta que atende em domicílio",

```

```

"arquiteta autônoma que recebe por projeto",
"redator freelancer que está pensando em abrir MEI",
"dono de uma pequena produtora de vídeo ME",
"contador que quer indicar a Indie para clientes",
"ilustrador que recebe de clientes no Reino Unido",
"profissional recém-demitido que vai começar a trabalhar como PJ",
]

def topic_styles(topic: dict[str, Any]) -> list[str]:
    """Return the question styles that apply to a topic."""
    return topic.get("styles", list(STYLES))

planned_calls = sum(len(topic_styles(t)) for t in TOPICS)
print(f"{len(TOPICS)} temas x estilos = {planned_calls} chamadas")
print(f"Pares brutos esperados: {planned_calls * Config.PAIRS_PER_CALL}")

```

24 temas x estilos = 140 chamadas
Pares brutos esperados: 700

1.6 5. Saída estruturada e prompt de geração

Como no notebook `langchain_structured_output.ipynb`, a saída do LLM é definida por modelos Pydantic. `with_structured_output` garante que cada chamada retorne uma lista válida de pares, sem parsing manual de texto.

```

[40]: class QAPair(BaseModel):
    """A single question and answer pair about Indie."""

    question: str = Field(
        description="Pergunta escrita como um cliente real escreveria"
    )
    answer: str = Field(
        description="Resposta da assistente virtual da Indie, fiel ao documento"
    )

class QABatch(BaseModel):
    """A batch of question and answer pairs about a single topic."""

    pairs: list[QAPair] = Field(
        description="Lista de pares de pergunta e resposta"
    )

```

O prompt de geração tem duas partes:

- **system:** o documento completo, como contexto, e as regras de qualidade para perguntas e respostas. As respostas devem ser fiéis, completas e seguir as diretrizes da seção 6. O prompt

também proíbe citar “o documento” na resposta.

- **human:** o tema, o estilo, as personas sorteadas e a lista de perguntas já geradas para o tema, que o modelo não pode repetir.

```
[41]: GENERATOR_SYSTEM_TEMPLATE = """Você é especialista em criar datasets de alta
    ↳ qualidade para o fine-tuning de assistentes virtuais.
    Sua tarefa é criar pares de pergunta e resposta sobre a Indie usando
    ↳ EXCLUSIVAMENTE as informações do documento de referência abaixo.

    <documento>
    {company_doc}
    </documento>

    Regras para as RESPOSTAS (escritas pela assistente virtual da Indie):
    1. Use somente fatos presentes no documento. Nunca invente produtos, taxas,
    ↳ preços, prazos, parceiros, números ou funcionalidades.
    2. Se a pergunta tratar de algo que o documento não cobre, a resposta deve
    ↳ dizer com clareza que não tem essa informação e indicar o canal de
    ↳ atendimento adequado.
    3. Siga as diretrizes do assistente virtual (seção 6 do documento): tom cordial,
    ↳ e direto, tratamento por "você", sem recomendação individual de investimento,
    ↳ nem planejamento tributário personalizado, nunca pedir senhas ou códigos,
    ↳ sem falar mal de concorrentes.
    4. As respostas devem ser completas e autocontidas, com 2 a 6 frases. Listas
    ↳ curtas são permitidas quando ajudam. Varie a abertura das respostas e não
    ↳ comece todas com saudação.
    5. Nunca mencione "o documento", "o texto" ou "as informações fornecidas". A
    ↳ assistente fala como a própria Indie.
    6. Escreva valores em reais no formato "R$ 39,90". Se fizer contas (ex.: taxa
    ↳ sobre um valor), confira o resultado.
    7. Não use informações de contato que não estejam no documento.

    Regras para as PERGUNTAS (escritas por clientes ou potenciais clientes):
    1. Escreva como uma pessoa real escreveria, no estilo pedido.
    2. Cada pergunta deve abordar um aspecto diferente do tema.
    3. Não repita nem parafraseie perguntas da lista de perguntas já geradas.
    4. Varie o início das frases, o vocabulário e o tamanho das perguntas."""

    GENERATOR_HUMAN_TEMPLATE = """Tema: {topic_name}
    Foco do tema: {topic_focus}

    Estilo das perguntas: {style_instruction}

    Personas sugeridas (use quando fizer sentido): {personas}

    Perguntas já geradas para este tema (NÃO repita):
    {existing_questions}
```

```
Gere exatamente {n_pairs} pares de pergunta e resposta."""
```

```
generation_prompt = ChatPromptTemplate.from_messages([
    ("system", GENERATOR_SYSTEM_TEMPLATE),
    ("human", GENERATOR_HUMAN_TEMPLATE),
])
```

1.7 6. Geração das perguntas e respostas

As funções abaixo implementam o loop de geração:

- `load_jsonl` e `append_jsonl` leem e gravam o checkpoint (`qna_bruto.jsonl`);
- `build_generation_inputs` monta as variáveis do prompt para um par (tema, estilo);
- `run_generation` percorre os estilos em rodadas. Em cada rodada, todos os temas são gerados em paralelo com `chain.batch`, e a rodada seguinte já recebe as perguntas das anteriores para evitar repetição. As chamadas que falham são repetidas até `MAX_RETRIES` vezes, e os pares (tema, estilo) já presentes no checkpoint são pulados.

```
[42]: def load_jsonl(path: Path) -> list[dict]:
    """Load a JSONL file into a list of dicts (empty list if missing)."""
    if not path.exists():
        return []
    with path.open(encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]

def append_jsonl(path: Path, records: list[dict]) -> None:
    """Append records to a JSONL file, one JSON object per line."""
    with path.open("a", encoding="utf-8") as f:
        for record in records:
            f.write(json.dumps(record, ensure_ascii=False) + "\n")

def format_existing_questions(
    records: list[dict], topic_id: str, limit: int
) -> str:
    """Format the latest questions already generated for a topic.

    Args:
        records: All records generated so far.
        topic_id: The topic identifier.
        limit: Maximum number of questions to include.

    Returns:
        A bullet list of questions, or a placeholder if there are none.
```

```

"""
questions = [r["question"] for r in records if r["topic"] == topic_id]
if not questions:
    return "(nenhuma ainda)"
return "\n".join(f"- {q}" for q in questions[-limit:])

def build_generation_inputs(
    topic: dict[str, Any],
    style_id: str,
    records: list[dict],
    company_doc: str,
    config: Config,
    rng: random.Random,
) -> dict[str, Any]:
    """Build the prompt variables for one (topic, style) generation call.

    Args:
        topic: The topic definition from 'TOPICS'.
        style_id: The question style key from 'STYLES'.
        records: All records generated so far (used to avoid repeats).
        company_doc: The company reference document.
        config: The pipeline configuration.
        rng: Random generator used to sample personas.

    Returns:
        A dict with the variables expected by 'generation_prompt'.
    """
    return {
        "company_doc": company_doc,
        "topic_name": topic["name"],
        "topic_focus": topic["focus"],
        "style_instruction": STYLES[style_id],
        "personas": "; ".join(rng.sample(PERSONAS, config.PERSONAS_PER_CALL)),
        "existing_questions": format_existing_questions(
            records, topic["id"], config.MAX_EXISTING_QUESTIONS
        ),
        "n_pairs": config.PAIRS_PER_CALL,
    }

def run_generation(
    chain: Any, company_doc: str, config: Config
) -> list[dict]:
    """Generate QnA pairs for every (topic, style), with checkpointing.

    Args:

```

*chain: A runnable that maps prompt variables to a 'QABatch'.
 company_doc: The company reference document.
 config: The pipeline configuration.*

Returns:

All generated records (including those loaded from checkpoint).

```

"""
raw_path = config.path(config.RAW_FILE)
records = load_jsonl(raw_path)
done = {(r["topic"], r["style"]) for r in records}
rng = random.Random(config.SEED + len(records))
print(f"Checkpoint: {len(records)} pares ja gerados.")

for style_id in tqdm(STYLES, desc="Estilos"):
    pending = [
        t
        for t in TOPICS
        if style_id in topic_styles(t) and (t["id"], style_id) not in done
    ]
    for attempt in range(1, config.MAX_RETRIES + 1):
        if not pending:
            break
        inputs = [
            build_generation_inputs(
                t, style_id, records, company_doc, config, rng
            )
            for t in pending
        ]
        results = chain.batch(
            inputs,
            config={"max_concurrency": config.MAX_CONCURRENCY},
            return_exceptions=True,
        )
        failed = []
        for topic, result in zip(pending, results):
            if isinstance(result, Exception) or not result or not result.
↪pairs:
                failed.append(topic)
                continue
        new_records = [
            {
                "id": f"{topic['id']}__{style_id}__{i}",
                "topic": topic["id"],
                "style": style_id,
                "question": pair.question.strip(),
                "answer": pair.answer.strip(),
            }

```

```

        for i, pair in enumerate(result.pairs)
    ]
    append_jsonl(raw_path, new_records)
    records.extend(new_records)
    done.add((topic["id"], style_id))
    if failed:
        errors = [r for r in results if isinstance(r, Exception)]
        print(
            f"[{style_id}] tentativa {attempt}: {len(failed)} falhas. "
            f"Ex.: {errors[0] if errors else 'resposta vazia'}"
        )
        time.sleep(5 * attempt)
        pending = failed
    if pending:
        print(f"[{style_id}] sem sucesso para: {[t['id'] for t in_
↵pending]}")

    print(f"Total de pares brutos: {len(records)}")
    return records

```

A célula abaixo monta a chain (prompt | llm estruturado), testa uma única chamada para validar o formato e depois executa a geração completa. Com o checkpoint, a célula pode ser reexecutada sem perder o que já foi gerado.

```

[43]: generation_chain = generation_prompt | generator_llm.with_structured_output(
        QABatch, method=config.STRUCTURED_METHOD
    )

    sample_batch = generation_chain.invoke(
        build_generation_inputs(
            TOPICS[0], "direta", [], COMPANY_DOC, config, random.Random(0)
        )
    )
    for pair in sample_batch.pairs[:2]:
        print(f"P: {pair.question}\nR: {pair.answer}\n")

```

P: Quando a Indie foi fundada e onde?

R: A Indie foi fundada em 2022, em São Paulo (SP).

P: Quem é o fundador da Indie?

R: O fundador da Indie é Matheus Silva, que também é o CEO e ex-desenvolvedor.

```

[44]: raw_records = run_generation(generation_chain, COMPANY_DOC, config)
    pipeline_stats["pares_brutos"] = len(raw_records)

```

Checkpoint: 590 pares já gerados.

Estilos: 0% | 0/6 [00:00<?, ?it/s]

```
[informal] tentativa 1: 4 falhas. Ex.: Error code: 429 - {'detail': 'Rate limit
exceeded: max 20 requests per 60s'}
[informal] tentativa 2: 4 falhas. Ex.: Error code: 429 - {'detail': 'Rate limit
exceeded: max 20 requests per 60s'}
[informal] tentativa 3: 4 falhas. Ex.: Error code: 429 - {'detail': 'Rate limit
exceeded: max 20 requests per 60s'}
[informal] sem sucesso para: ['previdencia', 'cobrancas', 'planos',
'regras_comerciais']
Total de pares brutos: 680
```

1.8 7. Limpeza e deduplicação

As funções abaixo aplicam os filtros automáticos de qualidade:

- `normalize_text`: remove espaços duplicados e padroniza a moeda como “R\$”;
- `basic_clean`: descarta perguntas e respostas curtas demais e remove duplicatas exatas (depois de normalizar maiúsculas e pontuação);
- `remove_near_duplicates`: vetoriza as perguntas com TF-IDF de n-gramas de caracteres (robusto a pequenas variações de escrita) e, para cada par de perguntas com similaridade de cosseno `DEDUP_THRESHOLD`, mantém só a primeira.

```
[45]: def normalize_text(text: str) -> str:
    """Normalize whitespace and currency formatting in a text."""
    text = re.sub(r"\bRRL\s*", "R$ ", text)
    text = re.sub(r"R\$(?=\d)", "R$ ", text)
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()

def basic_clean(records: list[dict], config: Config) -> pd.DataFrame:
    """Normalize texts, drop too-short pairs and exact duplicates.

    Args:
        records: Raw generated records.
        config: The pipeline configuration.

    Returns:
        A cleaned DataFrame with one row per QnA pair.
    """
    df = pd.DataFrame(records)
    df["question"] = df["question"].map(normalize_text)
    df["answer"] = df["answer"].map(normalize_text)

    too_short = (df["question"].str.len() < config.MIN_QUESTION_CHARS) | (
        df["answer"].str.len() < config.MIN_ANSWER_CHARS
    )
    print(f"Removidos por tamanho minimo: {too_short.sum()}")
```

```

df = df[~too_short]

question_key = (
    df["question"].str.lower().str.replace(r"^\w\s", "", regex=True)
    .str.split().str.join(" ")
)
duplicated = question_key.duplicated()
print(f"Removidas duplicatas exatas: {duplicated.sum()}")
return df[~duplicated].reset_index(drop=True)

def remove_near_duplicates(
    df: pd.DataFrame, threshold: float
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Remove questions that are near-duplicates of an earlier question.

    Args:
        df: The cleaned DataFrame.
        threshold: Cosine similarity above which two questions are
            considered near-duplicates.

    Returns:
        A tuple '(kept, removed_pairs)': the deduplicated DataFrame and a
        DataFrame listing each removed question with the kept one.
    """
    vectorizer = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5))
    similarity = cosine_similarity(vectorizer.fit_transform(df["question"]))
    keep = np.ones(len(df), dtype=bool)
    removed = []
    for i in range(len(df)):
        if not keep[i]:
            continue
        for j in np.where(similarity[i, i + 1 :] >= threshold)[0] + i + 1:
            if keep[j]:
                keep[j] = False
                removed.append(
                    {
                        "mantida": df.at[i, "question"],
                        "removida": df.at[j, "question"],
                        "similaridade": round(float(similarity[i, j]), 3),
                    }
                )
    print(f"Removidas quase-duplicatas: {len(removed)}")
    return df[keep].reset_index(drop=True), pd.DataFrame(removed)

```

```
[46]: clean_df = basic_clean(raw_records, config)
```

```
clean_df, near_duplicates = remove_near_duplicates(clean_df, config.
    ↪DEDUP_THRESHOLD)
pipeline_stats["apos_limpeza_e_deduplicacao"] = len(clean_df)

print(f"\nPares apos limpeza: {len(clean_df)}")
near_duplicates.head(10)
```

Removidos por tamanho minimo: 3

Removidas duplicatas exatas: 2

Removidas quase-duplicatas: 10

Pares apos limpeza: 665

```
[46]:
```

	mantida \		removida	similaridade
0	Quem pode abrir a Indie Conta PJ?			
1	Qual é a anuidade do cartão de crédito PJ nos ...			
2	Como faço para abrir um sinistro no Seguro Ren...			
3	Qual a diferença entre o Cofre Fiscal e as Cai...			
4	Por que eu deveria usar a Indie em vez do meu ...			
5	Por que eu deveria usar a Indie em vez do meu ...			
6	Por que eu deveria usar a Indie em vez do meu ...			
7	Por que eu deveria usar a Indie Contábil em ve...			
8	E se a Indie quebrar? O que acontece com o meu...			
9	Se minha empresa tiver mais de 5 sócios, posso...			
0	Quem não pode abrir a Indie Conta PJ?			0.963
1	Como funciona a anuidade do cartão de crédito ...			0.927
2	como faço pra abrir um sinistro no Seguro Rend...			0.969
3	Qual é a diferença entre o Cofre Fiscal e as C...			0.997
4	Por que eu deveria usar o Cofre Fiscal em vez ...			0.867
5	Por que eu deveria usar a Indie Contábil em ve...			0.922
6	Por que eu deveria usar a Indie Cobranças em v...			0.886
7	Por que eu deveria usar o Salário Indie em vez...			0.889
8	Se a Indie quebrar, o que acontece com o meu d...			0.880
9	Se eu tiver uma empresa com mais de 5 sócios, ...			0.943

1.9 8. Checagem de números

Taxas, preços e prazos são o tipo de informação que um LLM mais costuma “inventar”. A função `find_unknown_numbers` extrai os valores em BRL e os percentuais de cada resposta e sinaliza os que **não aparecem no documento**. Nem todo valor sinalizado é erro (pode ser uma conta correta, como 0,9% de US\$ 1.000), mas esses pares ganham prioridade na avaliação do juiz e na revisão manual.

```
[47]: NUMBER_PATTERN = re.compile(
    r"R\$\s?\d{1,3}(?:\.\d{3})*(?:,\d{1,2})?|\d+(?:,\d+)?\s?%"
)
```

```
def extract_numbers(text: str) -> set[str]:
    """Extract monetary values (R$) and percentages from a text."""
    return {re.sub(r"\s", "", m) for m in NUMBER_PATTERN.findall(text)}

def find_unknown_numbers(answer: str, known_numbers: set[str]) -> str:
    """Return the numbers of an answer that are not in the reference doc."""
    return ", ".join(sorted(extract_numbers(answer) - known_numbers))
```

```
[48]: DOC_NUMBERS = extract_numbers(normalize_text(COMPANY_DOC))
clean_df["numeros_fora_do_doc"] = clean_df["answer"].map(
    lambda answer: find_unknown_numbers(answer, DOC_NUMBERS)
)
flagged = clean_df[clean_df["numeros_fora_do_doc"] != ""]
pipeline_stats["pares_com_numeros_fora_do_doc"] = len(flagged)

print(f"Pares com numeros que nao estao no documento: {len(flagged)}")
flagged[["topic", "question", "answer", "numeros_fora_do_doc"]].head(10)
```

Pares com numeros que nao estao no documento: 0

```
[48]: Empty DataFrame
Columns: [topic, question, answer, numeros_fora_do_doc]
Index: []
```

1.10 9. Avaliação automática com LLM como juiz

Um segundo passe usa o LLM com temperatura 0 para comparar cada par com o documento, em lotes de JUDGE_BATCH_SIZE. O par é marcado como **inconsistente** quando a resposta:

- contém informação que contradiz o documento ou que não está nele;
- não responde à pergunta, ou é vaga ou truncada;
- viola as diretrizes da assistente (seção 6).

Os pares inconsistentes são descartados. Os vereditos também são salvos em checkpoint (qna_avaliacao_juiz.jsonl).

```
[49]: class JudgeVerdict(BaseModel):
    """The judge's verdict for a single QnA pair."""

    index: int = Field(description="Número do par avaliado, como indicado entre [ ]")
    consistent: bool = Field(
        description="true se a resposta é fiel ao documento, completa e segue as diretrizes"
    )
    issue: str = Field(
```

```

        description="Descrição curta do problema encontrado; string vazia se
        ↳ consistente"
    )

class JudgeBatch(BaseModel):
    """The judge's verdicts for a batch of QnA pairs."""

    verdicts: list[JudgeVerdict] = Field(description="Um veredito por par
    ↳ avaliado")

JUDGE_SYSTEM_TEMPLATE = """Você é um revisor rigoroso de datasets de
    ↳ fine-tuning.
    Avalie se cada par de pergunta e resposta sobre a Indie está correto em relação
    ↳ ao documento de referência.

    <documento>
    {company_doc}
    </documento>

    Marque consistent=false quando a resposta:
    - contiver qualquer informação que contradiga o documento ou que não esteja
    ↳ nele (valores, taxas, prazos, produtos, parceiros, canais, funcionalidades);
    - não responder ao que foi perguntado, estiver truncada ou for vaga demais;
    - violar as diretrizes da seção 6 (ex.: recomendar investimento específico,
    ↳ pedir senha, falar mal de concorrentes, fingir acesso à conta do cliente);
    - mencionar "o documento" ou "as informações fornecidas".

    Diferenças de redação, resumos corretos e cálculos corretos feitos a partir dos
    ↳ valores do documento são aceitáveis.
    Recusas educadas para perguntas fora do escopo são consistentes."""

JUDGE_HUMAN_TEMPLATE = """Avalie os pares abaixo e retorne um veredito para
    ↳ cada índice.

    {pairs}"""

judge_prompt = ChatPromptTemplate.from_messages(
    [("system", JUDGE_SYSTEM_TEMPLATE), ("human", JUDGE_HUMAN_TEMPLATE)]
)

def format_pairs_for_judge(batch: pd.DataFrame) -> str:
    """Format a batch of QnA pairs with numeric indexes for the judge."""
    return "\n\n".join(

```

```

        f"[{i}]\nPergunta: {row.question}\nResposta: {row.answer}"
        for i, row in enumerate(batch.itertuples())
    )

def run_judge(
    chain: Any, df: pd.DataFrame, company_doc: str, config: Config
) -> pd.DataFrame:
    """Judge every QnA pair against the reference doc, with checkpointing.

    Args:
        chain: A runnable that maps prompt variables to a 'JudgeBatch'.
        df: The cleaned DataFrame (must contain an 'id' column).
        company_doc: The company reference document.
        config: The pipeline configuration.

    Returns:
        A DataFrame with columns 'id', 'consistent' and 'issue'.
    """
    judge_path = config.path(config.JUDGE_FILE)
    judged_ids = {v["id"] for v in load_jsonl(judge_path)}
    pending = df[~df["id"].isin(judged_ids)]
    batches = [
        pending.iloc[i : i + config.JUDGE_BATCH_SIZE]
        for i in range(0, len(pending), config.JUDGE_BATCH_SIZE)
    ]
    print(f"Ja avaliados: {len(judged_ids)} | lotes pendentes: {len(batches)}")

    for attempt in range(1, config.MAX_RETRIES + 1):
        if not batches:
            break
        results = chain.batch(
            [
                {"company_doc": company_doc, "pairs": format_pairs_for_judge(b)}
                for b in batches
            ],
            config={"max_concurrency": config.MAX_CONCURRENCY},
            return_exceptions=True,
        )
        failed = []
        for batch, result in zip(batches, results):
            if isinstance(result, Exception) or not result:
                failed.append(batch)
                continue
            by_index = {v.index: v for v in result.verdicts}
            if not all(i in by_index for i in range(len(batch))):
                failed.append(batch)

```

```

        continue
    append_jsonl(
        judge_path,
        [
            {
                "id": row_id,
                "consistent": by_index[i].consistent,
                "issue": by_index[i].issue,
            }
            for i, row_id in enumerate(batch["id"])
        ],
    )
    if failed:
        print(f"Tentativa {attempt}: {len(failed)} lotes falharam.")
        time.sleep(5 * attempt)
        batches = failed

verdicts = pd.DataFrame(load_jsonl(judge_path))
return verdicts.drop_duplicates("id", keep="last")

```

```

[50]: judge_chain = judge_prompt | judge_llm.with_structured_output(
        JudgeBatch, method=config.STRUCTURED_METHOD
    )
verdicts = run_judge(judge_chain, clean_df, COMPANY_DOC, config)

judged_df = clean_df.merge(verdicts, on="id", how="left")
not_judged = judged_df["consistent"].isna().sum()
rejected = judged_df[judged_df["consistent"] == False] # noqa: E712
approved_df = judged_df[judged_df["consistent"] != False].copy() # noqa: E712

pipeline_stats["reprovados_pelo_juiz"] = len(rejected)
pipeline_stats["apos_juiz"] = len(approved_df)
print(f"Reprovados pelo juiz: {len(rejected)} | nao avaliados (mantidos): {len(not_judged)}")
print(f"Pares aprovados: {len(approved_df)}")
rejected[["topic", "question", "answer", "issue"]].head(15)

```

Já avaliados: 397 | lotes pendentes: 27
 Tentativa 1: 7 lotes falharam.
 Tentativa 2: 7 lotes falharam.
 Tentativa 3: 7 lotes falharam.
 Reprovados pelo juiz: 31 | nao avaliados (mantidos): 68
 Pares aprovados: 634

```

[50]:
      topic                                question \
14  conta_funcionalidades  Como funciona o pagamento de boletos na Indie ...
26      abre_mei  Tem alguma promoção para quem abre o MEI pelo ...

```

60	salario_indie	Quantos meses de salário a reserva atual cobre...
87	atendimento	Eu sou um cliente do plano Pro, como posso fal...
106	conta_funcionalidades	Como funciona o Pix Automático na Indie Conta PJ?
119	abre_mei	Posso abrir um MEI pelo Indie Abre MEI se já s...
124	notas	Qual a diferença entre o plano Essencial e o p...
135	contabil_mei	Quais são as diferenças entre os planos da Ind...
137	contabil_mei	Além da DASN-SIMEI, o que mais está incluído n...
140	contabil_me	Quais são as diferenças nos serviços oferecido...
174	planos	Quais são as diferenças entre os planos em rel...
179	regras_comerciais	Indie Contábil é a única opção que tenho para ...
225	global	Não consigo ver os dados de recebimento na min...
237	contabil_mei	O que acontece se eu não entregar a DASN-SIMEI...
242	contabil_me	Não consigo acessar o atendimento do meu conta...

answer \

14	Na Indie Conta PJ, é possível pagar boletos e ...
26	Sim! Quem abre o MEI pelo Indie Abre MEI ganha...
60	O app mostra quantos meses de salário a reserv...
87	Se você é cliente do plano Premium, pode falar...
106	O Pix Automático da Indie Conta PJ permite que...
119	Sim, você pode abrir um MEI pelo Indie Abre ME...
124	No plano Essencial, você pode emitir até 5 not...
135	Os planos da Indie Contábil para MEI oferecem ...
137	Na assinatura da Indie Contábil para MEIs, alé...
140	A Indie Contábil oferece diferentes níveis de ...
174	No plano Essencial, você tem um cartão de créd...
179	A Indie Contábil é uma assinatura separada e o...
225	Para visualizar os dados de recebimento no Ind...
237	Se você não entregar a DASN-SIMEI dentro do pr...
242	Se você não consegue acessar o atendimento do ...

issue

14	O plano Essencial oferece 10 boletos pagos por...
26	A promoção mencionada está incorreta; quem abr...
60	A resposta não fornece a informação específica...
87	A resposta menciona o plano Premium, mas a per...
106	O Pix Automático não é mencionado no documento...
119	A resposta não menciona que o Indie Abre MEI n...
124	O plano Premium não inclui um certificado digi...
135	A resposta menciona que a DASN-SIMEI custa R\$...
137	A resposta menciona que a declaração anual est...
140	A resposta não menciona que a Declaração Anual...
174	O cartão de crédito PJ no plano Essencial não ...
179	A Indie Contábil é uma opção de contabilidade,...
225	A resposta menciona 'dados de recebimento na m...
237	A resposta menciona penalidades e multas, mas ...
242	A resposta não menciona que o atendimento do c...

1.11 10. Estatísticas da base

As tabelas abaixo mostram a distribuição dos pares aprovados por tema e por estilo e o tamanho das respostas, para confirmar que a base está equilibrada e sem respostas truncadas. Guarde esses números para o relatório.

```
[51]: print(f"Total de pares aprovados: {len(approved_df)}")
      if len(approved_df) < config.TARGET_MIN_EXAMPLES:
          print(
              "ATENCAO: abaixo de 500 pares. Aumente PAIRS_PER_CALL ou adicione "
              "estilos e reexecute a geracao (o checkpoint preserva o que ja existe)."
          )

      distribution = pd.crosstab(approved_df["topic"], approved_df["style"],
                               margins=True)
      distribution
```

Total de pares aprovados: 634

```
[51]: style
      topic
      abre_mei          5          4          4          5          4
      atendimento        5          5          4          5          5
      caixinhas          5          5          5          5          5
      cartoes           4          4          5          4          3
      cobranças         5          5          5          0          4
      cofre_fiscal       5          5          5          4          4
      conta_abertura     5          5          4          5          5
      conta_funcionalidades 5          4          4          5          5
      contabil_me        5          4          5          5          5
      contabil_mei       5          3          5          5          4
      empresa           5          5          3          4          5
      fora_escopo        0          0          5          5          0
      global            5          5          5          5          5
      limites_assistente 5          4          3          5          5
      nao_oferece        5          5          5          5          4
      notas             5          4          5          5          4
      planos            5          4          4          0          4
      previdencia       5          5          5          0          4
      regras_comerciais  5          4          5          0          4
      regulacao         5          5          5          5          5
      salario_indie      5          5          4          5          4
      seguranca         5          5          5          5          5
      seguro            4          5          5          4          5
      suporte           5          5          5          4          5
      All              113         105         110         95         103

      style          suporte  All
```

topic		
abre_mei	5	27
atendimento	5	29
caixinhas	5	30
cartoes	5	25
cobrancas	5	24
cofre_fiscal	5	28
conta_abertura	5	29
conta_funcionalidades	5	28
contabil_me	4	28
contabil_mei	4	26
empresa	5	27
fora_escopo	0	10
global	4	29
limites_assistente	5	27
nao_oferece	5	29
notas	5	28
planos	5	22
previdencia	2	21
regras_comerciais	5	23
regulacao	5	30
salario_indie	5	28
seguranca	4	29
seguro	5	28
suporte	5	29
All	108	634

```
[52]: approved_df.assign(
        tamanho_pergunta=approved_df["question"].str.len(),
        tamanho_resposta=approved_df["answer"].str.len(),
    )["tamanho_pergunta", "tamanho_resposta"].describe().round(0)
```

```
[52]:      tamanho_pergunta  tamanho_resposta
count                634.0                634.0
mean                  87.0                252.0
std                   38.0                 83.0
min                   25.0                 61.0
25%                   59.0                190.0
50%                   76.0                252.0
75%                  103.0                309.0
max                   245.0                530.0
```

1.12 11. Revisão manual (amostra de 10%)

A célula abaixo sorteia uma amostra **estratificada por tema** de 10% dos pares aprovados e salva o arquivo `amostra_revisao_manual.csv` (separador `;`, que abre direto no Excel ou no Google Sheets).

Como revisar: para cada linha, preencha a coluna `decisao` com:

- OK: o par está correto (linha em branco também conta como OK);
- EDITAR: preencha `pergunta_corrigida` e/ou `resposta_corrigida`;
- REMOVER: o par será excluído da base.

Confira com atenção as linhas com `numeros_fora_do_doc` preenchido. Depois, salve o arquivo como `amostra_revisao_manual_revisada.csv` e envie na célula seguinte.

```
[53]: def export_review_sample(df: pd.DataFrame, config: Config) -> pd.DataFrame:
    """Export a topic-stratified sample of pairs for manual review.

    Args:
        df: The approved DataFrame.
        config: The pipeline configuration.

    Returns:
        The exported sample as a DataFrame.
    """
    sample = df.groupby("topic").sample(
        frac=config.REVIEW_FRACTION, random_state=config.SEED
    )
    sample = sample[["id", "topic", "style", "question", "answer",
↪ "numeros_fora_do_doc"]]
    sample = sample.assign(
        decisao="", pergunta_corrigida="", resposta_corrigida="", comentario=""
    )
    sample.to_csv(
        config.path(config.REVIEW_FILE), sep=";", index=False,
↪ encoding="utf-8-sig"
    )
    return sample

def apply_manual_review(
    df: pd.DataFrame, reviewed_path: Path
) -> tuple[pd.DataFrame, Optional[pd.Series]]:
    """Apply the group's manual review decisions to the dataset.

    Args:
        df: The approved DataFrame.
        reviewed_path: Path to the reviewed CSV file.

    Returns:
        A tuple '(df, summary)': the updated DataFrame and the count of
        each decision (None if the reviewed file does not exist).
    """
    if not reviewed_path.exists():
```

```

        print(f"Arquivo revisado nao encontrado em {reviewed_path}.")
        return df, None

    review = pd.read_csv(reviewed_path, sep=";", encoding="utf-8-sig",
        dtype=str)
    review = review.fillna("")
    review["decisao"] = review["decisao"].str.strip().str.upper().replace(" ",
        "OK")

    df = df.copy()
    for row in review[review["decisao"] == "EDITAR"].itertuples():
        mask = df["id"] == row.id
        if row.pergunta_corrigida.strip():
            df.loc[mask, "question"] = normalize_text(row.pergunta_corrigida)
        if row.resposta_corrigida.strip():
            df.loc[mask, "answer"] = normalize_text(row.resposta_corrigida)

    removed_ids = set(review.loc[review["decisao"] == "REMOVER", "id"])
    df = df[~df["id"].isin(removed_ids)].reset_index(drop=True)
    return df, review["decisao"].value_counts()

```

```

[54]: review_sample = export_review_sample(approved_df, config)
pipeline_stats["amostra_revisao_manual"] = len(review_sample)
print(f"Amostra para revisao: {len(review_sample)} pares -> {config.path(config.
    REVIEW_FILE)}")

try:
    from google.colab import files

    files.download(str(config.path(config.REVIEW_FILE)))
except ImportError:
    pass

```

Amostra para revisao: 65 pares ->
/content/drive/MyDrive/PUC_GenAI_Indie/amostra_revisao_manual.csv

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

Depois de revisar a planilha, execute a célula abaixo e envie o arquivo revisado. Se você salvou o arquivo direto na pasta do projeto no Drive com o nome `amostra_revisao_manual_revisada.csv`, o upload é pulado.

```

[55]: reviewed_path = config.path(config.REVIEWED_FILE)
if not reviewed_path.exists():
    from google.colab import files

    print("Envie o arquivo CSV revisado:")

```

```

uploaded = files.upload()
reviewed_path.write_bytes(next(iter(uploaded.values()))))

final_df, review_summary = apply_manual_review(approved_df, reviewed_path)
if review_summary is not None:
    pipeline_stats["revisao_manual"] = review_summary.to_dict()
    print(review_summary)
print(f"\nPares apos a revisao manual: {len(final_df)}")

```

decisao

OK 57

Name: count, dtype: int64

Pares apos a revisao manual: 634

1.13 12. Exportação no formato de conversa (messages)

Cada par vira uma conversa com três mensagens: **system** (o prompt de sistema da seção 7 do documento), **user** (a pergunta) e **assistant** (a resposta). Esse é o mesmo formato do dataset usado em aula. A ordem dos exemplos é embaralhada, e dois arquivos são salvos:

- `indie_knowledge_base.jsonl`: dataset de treinamento, só com **messages**;
- `indie_qna_com_metadados.jsonl`: a mesma base com tema, estilo e id, para rastreabilidade.

A validação no final relê o `.jsonl` e confere a estrutura de todas as linhas.

```

[56]: def to_chat_example(question: str, answer: str, system_prompt: str) -> dict:
        """Convert a QnA pair into the chat 'messages' training format."""
        return {
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": question},
                {"role": "assistant", "content": answer},
            ]
        }

def write_datasets(df: pd.DataFrame, system_prompt: str, config: Config) -> None:
    """Write the training JSONL and the metadata JSONL files."""
    shuffled = df.sample(frac=1, random_state=config.SEED).
    reset_index(drop=True)
    with config.path(config.FINAL_FILE).open("w", encoding="utf-8") as f_train, \
        config.path(config.METADATA_FILE).open("w", encoding="utf-8") as \
    f_meta:
        for row in shuffled.itertuples():
            example = to_chat_example(row.question, row.answer, system_prompt)
            f_train.write(json.dumps(example, ensure_ascii=False) + "\n")

```

```

        f_meta.write(
            json.dumps(
                {"id": row.id, "topic": row.topic, "style": row.style,
↪**example},
                ensure_ascii=False,
            )
            + "\n"
        )

def validate_dataset(path: Path) -> int:
    """Validate that every line is a well-formed system/user/assistant chat.

    Args:
        path: Path to the training JSONL file.

    Returns:
        The number of valid examples.

    Raises:
        ValueError: If any line is malformed.
    """
    expected_roles = ["system", "user", "assistant"]
    with path.open(encoding="utf-8") as f:
        for line_number, line in enumerate(f, start=1):
            messages = json.loads(line)["messages"]
            roles = [m["role"] for m in messages]
            if roles != expected_roles or not all(m["content"].strip() for m in
↪messages):
                raise ValueError(f"Linha {line_number} invalida: {roles}")
    return line_number

```

```

[57]: write_datasets(final_df, SYSTEM_PROMPT, config)
n_examples = validate_dataset(config.path(config.FINAL_FILE))
pipeline_stats["dataset_final"] = n_examples
print(f"Dataset valido com {n_examples} exemplos: {config.path(config.
↪FINAL_FILE)}")

for example in random.Random(config.SEED).sample(load_jsonl(config.path(config.
↪FINAL_FILE)), 3):
    print("-" * 80)
    print("USER:", example["messages"][1]["content"])
    print("ASSISTANT:", example["messages"][2]["content"])

```

Dataset valido com 634 exemplos:
/content/drive/MyDrive/PUC_GenAI_Indie/indie_knowledge_base.jsonl

USER: Posso usar um celular simples para o Tap to Pay ou preciso de um smartphone mais avançado?

ASSISTANT: Para usar o Tap to Pay, seu celular precisa ter suporte a NFC, o que está disponível em dispositivos Android com NFC e em iPhones a partir do modelo XS. Portanto, se o seu celular não possui essa funcionalidade, você não conseguirá usar o Tap to Pay.

USER: Estou pensando em usar as Caixinhas, mas qual é o rendimento mesmo?

ASSISTANT: O rendimento das Caixinhas é de 100% do CDI no plano Essencial, 103% no Pro e 105% no Premium. É uma boa maneira de fazer seu dinheiro render!

USER: O que muda no funcionamento do Cofre Fiscal entre os planos Pro e Premium?

ASSISTANT: No plano Pro, a porcentagem reservada para o Cofre Fiscal é sugerida e recalculada automaticamente a cada mês, e é incluído o Simulador de Fator R. Já no plano Premium, além de tudo do Pro, há a integração completa com a Indie Contábil, que permite que a apuração do contador atualize automaticamente o Cofre Fiscal, oferecendo mais eficiência na gestão financeira.

1.14 13. Resumo para o relatório e download

O dicionário abaixo traz o funil completo de construção da base (gerados → limpeza → juiz → revisão manual → final). Ele é salvo em `resumo_pipeline.json` para ser usado no relatório em PDF.

```
[58]: config.path("resumo_pipeline.json").write_text(
        json.dumps(pipeline_stats, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(pipeline_stats, ensure_ascii=False, indent=2))

    try:
        from google.colab import files

        for filename in (config.FINAL_FILE, config.METADATA_FILE, "resumo_pipeline.
        ↳json"):
            files.download(str(config.path(filename)))
    except ImportError:
        pass
```

```
{
  "pares_brutos": 680,
  "apos_limpeza_e_deduplicacao": 665,
  "pares_com_numeros_fora_do_doc": 0,
  "reprovados_pelo_juiz": 31,
  "apos_juiz": 634,
  "amostra_revisao_manual": 65,
  "revisao_manual": {
    "OK": 57
  },
  }
```

```
"dataset_final": 634
}
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
<IPython.core.display.Javascript object>
```